Ansible Automation - Use Cases

A reference of the use cases by which Ansible Automation Platform is commonly implemented, by domain. Each use case is described in the same format (Description, Collection, Key content, Notes) and mapped to Red Hat Certified and Validated content.

3 solution use cases · 2 product use cases · generated 2026-10-21 · community reference, not an official Red Hat publication

Contents

1. Edge
    - Solution use cases (3)
        - Manage device fleets with Red Hat Edge Manager
        - Deploy MicroShift at the edge
        - Build and update edge OS images
    - Product use cases
        - Red Hat Device Edge (MicroShift) (1)
        - Red Hat Edge Manager (Flight Control) (1)

Edge

Fleet management for edge devices and remote sites.

Solution use cases

Use Case: Manage device fleets with Red Hat Edge Manager

Description	Manage thousands of edge devices as fleets instead of one by one. Red Hat Edge Manager (based on Flight Control) keeps each device on the operating system image and configuration declared for its fleet; Ansible creates the fleets, approves new devices and changes the desired state, and Edge Manager rolls it out to the devices.
Collection	`redhat.edge_manager` (Certified)
Key content	• `redhat.edge_manager.flightctl_resource` — Manage Flight Control resources • `redhat.edge_manager.flightctl_resource_info` — Get information about Flight Control resources • `redhat.edge_manager.flightctl_certificate_management` — Manage approving or denying certificate signing or enrollment requests • `redhat.edge_manager.flightctl_enrollment_config_info` — Get information about enrollment configuration for Flight Control. • `redhat.edge_manager.flightctl_image_builder` — Manage Flight Control Image Builder resources • `redhat.edge_manager.flightctl` — Returns Ansible inventory using Flight Control as source. • `redhat.edge_manager.flightctl_console` — Connect to Flight Control managed devices
Notes	• Roll out in stages: update a canary fleet or label first, then widen the change after status checks. • Pair with Deploy MicroShift at the edge when the devices run Kubernetes workloads. • Use Event-Driven Ansible to trigger an enrollment approval or a rollback job when device status events arrive from your monitoring system.

Use Case: Deploy MicroShift at the edge

Description	Run a small, supported Kubernetes platform on single-node edge sites. The collection installs MicroShift on RHEL either directly from RPM packages or inside an rpm-ostree image, and deploys application manifests to the running node, so each site gets the same platform and workloads.
Collection	`edge.microshift` (Validated), `infra.osbuild` (Validated)

Key content	• `edge.microshift.rpm_install` — Install Microshift on a RPM based system • `edge.microshift.image_builder` — Build an operating system image using osbuild composer with microshift and depencies installed • `edge.microshift.app` — Deploy an application onto a system running microshift • `edge.microshift.microshift_rpm_install` — Run Microshift rpm install role • `edge.microshift.microshift_image_build` — Run microshift image builder role • `edge.microshift.microshift_app` — Deploy application on microshift
Notes	• Combine with Manage device fleets with Red Hat Edge Manager when the number of sites makes direct SSH access impractical. • For Kubernetes workflows on larger clusters, see the Kubernetes use cases instead; MicroShift targets small single-node deployments. • Keep the pull secret out of Git, and rotate it with the credential store.

Use Case: Build and update edge OS images

Description	Build the operating system for edge devices as a versioned image, publish it, and move devices to the next version in a controlled way. Updates are image-based (rpm-ostree), so devices change in one step to a known version.
Collection	`infra.osbuild` (Validated), `ansible.posix` (Certified)
Key content	• `infra.osbuild.setup_server` — Setup osbuild compose server • `infra.osbuild.builder` — The main entry point for the builder role. • `infra.osbuild.populate_aap` — This role automates the process of populating AAP with infra.osbuild content • `infra.osbuild.update_system` — Update a rpm-ostree based system with the latest image. • `infra.osbuild.system_info` — Gather information from a rpm-ostree based system • `ansible.posix.rpm_ostree_upgrade` — Manage rpm-ostree upgrade transactions • `ansible.posix.rhel_rpm_ostree` — Ensure packages exist in a RHEL for Edge rpm-ostree based system
Notes	• The previous deployment stays on an rpm-ostree system after an update, so a device can be returned to it. Test the rollback on a spare device before the first production wave. • Build MicroShift into the image with Deploy MicroShift at the edge. • For large fleets, define the image in Manage device fleets with Red Hat Edge Manager and let Edge Manager roll it out. • General-purpose image builds are described in Build golden images with image builder.

Product use cases

Grouped from the modules of each certified or validated collection. Key content lists the components each use case supports, with the modules that manage them (create, update, delete) and query them (_info).

Red Hat Device Edge (MicroShift) `edge.microshift`

Validated Red Hat · version 1.0.3 · updated 2023-08-03 · 0 modules → 1 use cases

Use Case: Install MicroShift and deploy applications

Description	Install MicroShift from RPMs, build OS images with MicroShift using image builder and deploy applications onto it.
Collection	`edge.microshift` (Validated, Red Hat)
Key content	**Install from RPM** role `rpm_install` **RPM install playbook** playbook `microshift_rpm_install` **Image builder** role `image_builder` **Image build playbook** playbook `microshift_image_build` **Application deployment** role `app` **Application deployment playbook** playbook `microshift_app`

Red Hat Edge Manager (Flight Control) `redhat.edge_manager`

Certified Red Hat · version 1.7.0 · updated 2026-09-07 · 6 modules → 1 use cases

Use Case: Manage edge devices with Flight Control

Description	Manage Flight Control resources, image builds, certificate requests and enrollment configuration, connect to managed devices and build inventory.
Collection	`redhat.edge_manager` (Certified, Red Hat)
Key content	**Flight Control resources** `flightctl_resource`, `flightctl_resource_info` **Image builder** `flightctl_image_builder`, `flightctl_image_builder_info` **Certificate requests** `flightctl_certificate_management` **Enrollment configuration** `flightctl_enrollment_config_info` **Device console connection** connection `flightctl_console` **Flight Control inventory** inventory `flightctl`